# Low-Latency Asynchronous FIFO Buffers

J.T. Yantchev[1], C.G. Huang[2], M.B. Josephs[3], I.M. Nedelchev[4]

[1] Department of Computer Science,
University of Adelaide, SA 5005, Australia
e-mail: jty@cs.adelaide.edu.au

[2] Oxford University Computing Laborator
Parks Road, Oxford, OX1 3QD, UK
e-mail: {jty, huang}@comlab.ox.ac.uk

[3] Centre for Concurrent Systems and VLSI,
School of CISM, South Bank University,
103 Borough Road, London SE1 0AA, UK
e-mail: josephmb@vax.sbu.ac.uk

[4] Department of Electronic and Electrical Eng,
University of Surrey,
Guildford, Surrey, UK.
e-mail: i.nedelchev@ee.surrey.ac.uk

**Abstract**

*A parallel asynchronous implementation of a FIFO buffer is described and compared with the coneventional alternative asynchronous implementation, Sutherland's micropipeline. The parellel design has the potential for significant reductions in propagation delay at the cost of insignificant increases in cycle-time (i.e. reduced throughput) and area. Although in certain applications, e.g. DSP, only high throughput may be important, in others, e.g. packet switching, throughput and propagation delay both matter*

*We consider the parallel design to be most useful as part of the interface circuitry required by devices that asynchronously exchange data in bursts over inter-chip communication wires and use a single acknowledge signal for each burst of data. In particular, a high-throughput multiple-burst signalling scheme is supported, in which a second burst of data is transmitted at the same time as the previous burst is acknowledged, effectively increasing the overal throughput.*

## 1 Introduction

An $N$-place FIFO buffer operates as a flow-through first-in-first-out memory. It can input and buffer up to N data words before any are output. An attractive asynchronous implementation [7] involves a linear array or pipeline of cells of length $N$. The cycle-time, and hence the throughput (or data going through in unit time in steady state), of the pipeline is determined by the speed of its basic cell and is independent of the length $N$ of the pipeline. However, its propagation delay, or *latency*, is proportional to N.

A number of applications, particularly those where FIFO buffers are used to both store and process data, are relatively insensitive to increased latency. However, increases in latency incurred just for the purpose of buffering data are highly undesirable in other applications, e.g. high-performance packet switching, with little or no processing of data. (An example of the latter is the *mad postman* bit-serial switching [1,2,3], which stores and processes only one bit at a time per switch; the inclusion of an input buffer of length $N$, where $N$ can be several bytes, will have a significant effect on the latency of the switch.)

The simplest solution of providing no buffering at all on the data-path is not always feasible. For example, an inter-chip signalling scheme where several bits can be pipelined on each interconnecting wire, or where only one reverse acknowledgement is generated per burst of data consisting of several bits, necessitates buffering at each input link to ensure that all the bits of data are received correctly. The advantage of this scheme is that it increases the throughput of the link.

This paper explores the use of parallel FIFO buffers to avoid the sacrifice of latency for the purposes of increasing throughput. Our design basically maps into hardware the circular array implementation of a buffer, familiar to programmers, by connecting up registers in parallel rather than as a pipeline; this is sometimes called a *wagging* buffer [9]. A parallel FIFO design has also been independently developed and proposed recently in [10]. The resulting FIFO buffers can be considered to have latency $O(1)$, or $O(k \log N)$, where k is a very small constant.



0-8186-7098-3/95 $04.00 © 1995 IEEE

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

In the remainder of this paper, we first review inter-chip signalling schemes, as might be used for communication of packets through an asynchronous packet-switched network. Throughput is improved by acknowledging bursts of data rather than individual words. Since the receiver may not always be ready to process each incoming burst as soon as it arrives, buffering has to be provided. Sutherland's micropipeline is considered, before we go on to describe in detail the parallel design. We show how this has a much improved latency but can still support a high-throughput multiple-burst signalling scheme, in which a second burst of data is transmitted at the same time as the previous burst is acknowledged. In drawing our conclusions, we quantify the trade-offs in terms of latency, throughput and area between the two alternative implementations of the FIFO buffer.

## 2. Asynchronous Packet Switching Devices and Interchip Signaling

Asynchronous implementation of packet switches has been shown to be very attractive in improving the latency of packet propagation [4,5,6], both in the implementation of fine-grain switching fabric for cross-bar switches and of large-grain packet routing nodes for multi-computer networks. Some advanced packet switching techniques, e.g. mad postman, can in principle achieve node-through delay that is lower than the cycle time of the inter-switch links, a lower limit that cannot be reached using conventional synchronous techniques. The asynchronous implementation of packet switches is attractive also because, as asynchronous circuits are allowed to operate at their own rate, packet propagation rate will be determined only by the performance of the individual switches, not limited by their global arrangement and network size. Delay-insensitive inter-node signalling will considerably simplify the composition of large packet switching networks as inter-node wires can be made arbitrarily long. Increasing the lengths of the interconnecting wires will gracefully increase inter-node delays, but will not affect correct operation.

Low propagation latency, however, is not the only important performance parameter of packet switching networks. High throughput is equally and in some cases even more important than low latency. Throughput is determined by many factors, but perhaps most by the transmission rate of the inter-node links. One strong objection that has been raised against the use of delay-insensitive signalling for inter-node transmission is the dependency between the delay on the wire and the transmission rate. The time to transmit one unit of data, e.g. one bit on a bit-serial link, is determined by the delay in the forward propagation of request/data and the delay in propagating a reverse acknowledgement signal before the next unit of data can be sent, Figure 1. This time obviously grows with the length of the wire and the transmission rate may become unacceptably low when compared with other delay-sensitive signalling techniques (which may even allow more than one bit of data to be in transmission on the same wire at the same time [6]).

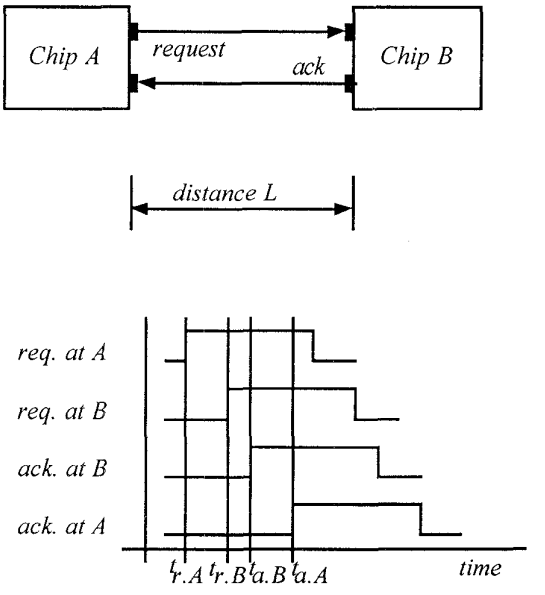



**Fig 1 Delay-insensitive inter-chip signaling.**

Figure 1 illustrates the dependency of the transmission rate on the wire length $L$ when delay-insensitive signalling [8] is used. The timing diagram illustrates the two signals, *request* and *acknowledge*, at nodes $A$ and $B$. The delay between transmitting two successive bits of data is at least $d = (t_{a.A} - t_{r.A})$, which is proportional to the wire length $L$. The maximum transmission rate is of course no more than 1/d.

This leads us to investigate ways of implementing asynchronous packet switches using *delay-sensitive* signalling for inter-chip transmission in order to achieve maximum transmission rate and throughput, while internally (where wires are short) still using delay-insensitive signalling for ease of design. Delay-sensitive transmission will allow both more than one bits to be in transmission on a wire at the same time and a whole burst of $N$ words of data, where $N>1$, to be sent by the sender before an acknowledge signal is received, Figure 2 below



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

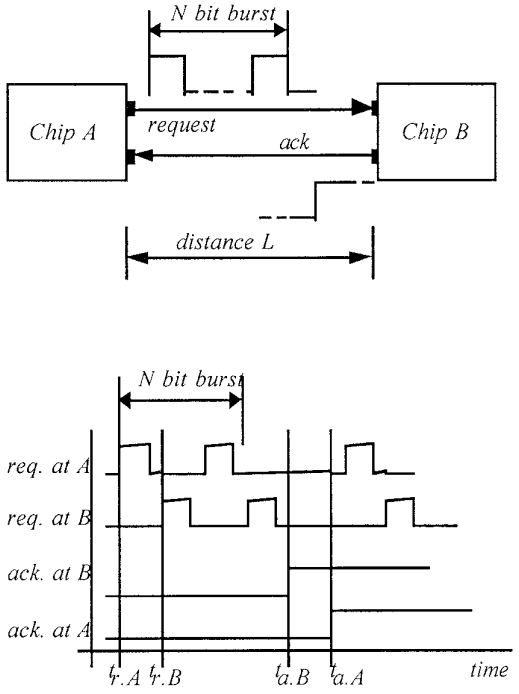



**Figure 2. Delay-sensitive interchip transmission of *N*-bit bursts**

Not every bit is acknowledged and several bits may be pipelined on the same wire at the same time. Therefore the data transmission rate is decoupled from the wire length L and can be increased significantly.

Delay-sensitive signalling is not as safe and resilient to signal degradation as its delay-insensitive counterpart and requires great care in both circuit design and system composition. Though difficult, these implementation problems are not insurmountable and commercial devices using such signalling have been built [5, 6]. The transmission system for delay-sensitive signalling can be decomposed into a sender, S, a receiver, R, and interconnecting wires, W, Figure 3.

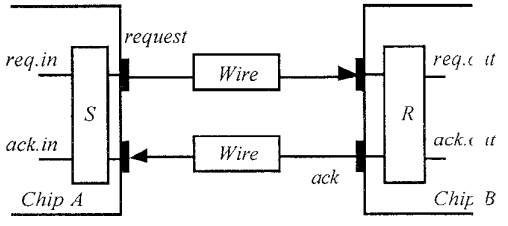


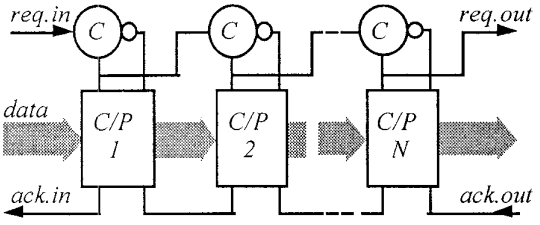

**Fig 3 Delay-sensitive transmission system.**

The sender S has a delay-insensitive input end (left-hand side) and a delay-sensitive output end. The latter only waits for acknowledgement that the whole burst has been received by R and a new burst may be sent. The wires W are carefully controlled to not introduce any errors due to signal degradation or overlap. The receiver R has a delay-sensitive input end and a delay-insensitive output end. The former sends a reverse acknowledge only after all N data items have been output, i.e. the receiver is empty and so ready to accommodate another burst. The receiver must be able to buffer all $N$ bits of data of each burst as well as being sufficiently fast to latch in all bits without errors. The whole system comprising $S$, $W$, and $R$, operates as a delay-insensitive N-place FIFO buffer.

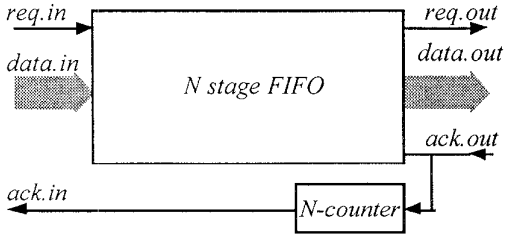
In this paper we will only consider the design of the receiver $R$; the sender $S$ is relatively straightforward to design. The receiver will need to use a FIFO buffer of capacity $N$ and low cycle-time to meet the above requirements, unless the data processing circuitry itself is already able to provide buffer space on the data-path. Use of an asynchronous implementation of the FIFO buffer will even allow the sender to transmit individual words at irregular intervals, i.e. both slow and fast bursts are accommodated.

## 3 Linear Pipeline FIFO Buffer

Sutherland [7] has proposed that $N$-place FIFO buffers can be implemented as elastic pipelines of length $N$, as in Figure 4(a).




*a) N-stage elastic pipeline*




*b) FIFO buffer for burst transmission*

**Fig 4 Pipeline FIFO buffer for burst transmission.**



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

Each bit of data must propagate through all stages of the pipeline before it appears at its output. To generate a single reverse acknowledge signal *ack.in* per burst, an *N*-counter connected to the *ack.out* signal of the right-hand side of the pipeline is used, Fig. 4(b). This ensures that *ack.in* is sent only after the last bit of data has left the FIFO and it is therefore empty and ready to accommodate the next burst.

What are the implications of inserting an extra FIFO on the packet propagation path, as illustrated in Figure 5?

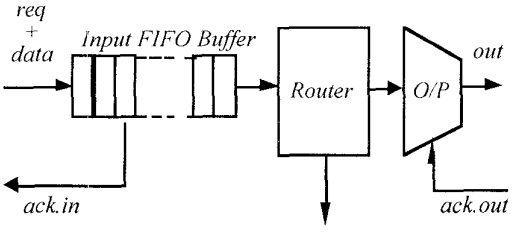



**Fig 5 Simplified datapath of a packet switch - from input to output.**

The extra buffer will inevitably increase the node-through delay and therefore the overall latency. The latency of the elastic pipeline is proportional to N and therefore the node-through delay will also be proportional to N. In our attempt to improve the node throughput by using delay-sensitive transmission we have gone to the opposite extreme of significantly increasing the node latency!

As the increased node latency is due to use of the pipeline, then perhaps an optimal solution will be reached if we can find a way of implementing FIFO buffers of very low latency allowing a data item to be read out as soon as it is latched in. As before, the low latency FIFO must be of low cycle-time in order to be fast enough to latch all bits of data as they arrive and not sacrifice throughput.

## 4 Low Latency Parallel FIFO Buffer

The low-latency FIFO is constructed as parallel strcuture of *N* buffers, each of capacity 1. The design is naturally decomposed into two components, a datapath and a control circuit, Figure 6.

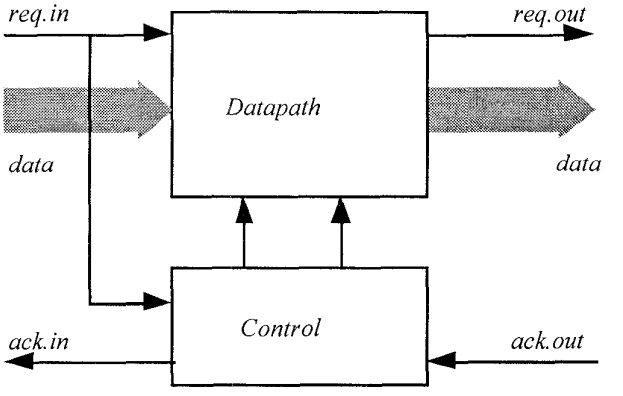



**Fig 6 Decomposition of the FIFO into a data path and a control circuit.**

Such a decomposition is very common, but what is unusual here is that the input request control signal, req.in, is processed together with the data and the output request control signal, req.out, is generated by the data-path rather than the control circuit. This is indeed unusual as, with bundled data signalling, it is the level of each data wire that is significant, in contrast to the transition of a request wire (which signifies that data is stable). The reason for doing this is to reduce circuit size and increase speed, as will be explained later. For now, we consider the two components in more detail.

**Data-path.** To construct an *N*-place FIFO buffer with latency that does not depend on *N* we need to compose the data storage elements in parallel, Figure 7, rather than in series, as was the case for the elastic pipeline of Figure 4.

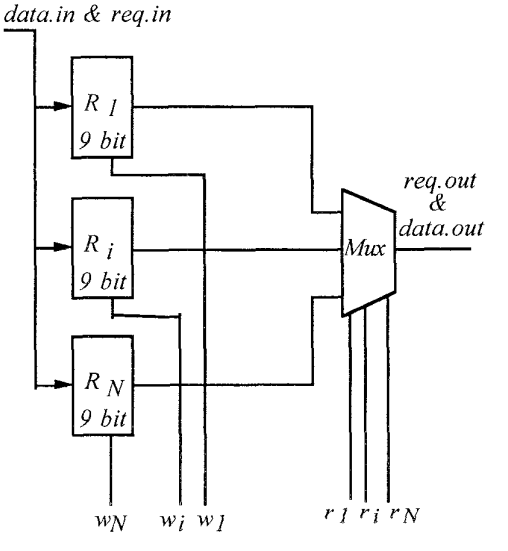



**Fig 7 Low latency FIFO - datapath.**



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

Writing into the buffer is concurrent with reading and each data word can be read out as soon as it is written in. The multiplexer, *Mux*, has at worst a $k\ logN$ delay, where $k$ can be very small, and the delay of the whole FIFO buffer is about the delay through a storage register and therefore very small.

**Control circuit.** The control circuit comprises two $N$-counters, *Cw* and *Cr*, Figure 8.

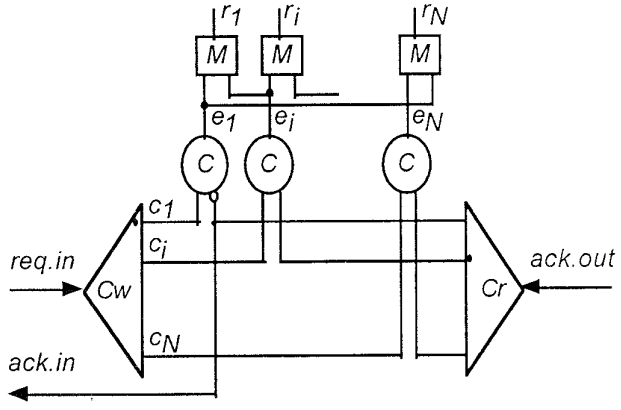



**Fig 8 Parallel FIFO - control circuit.**

At the beginning of each burst Cw is in its initial state; the $i$-th request signal *req.in* causes a transition on the $c_i$ output of *Cw*, which is then used to write into register $R_i$ the data on the data wires data.in. As it is much easier to construct registers with return-to-zero clock lines (both for latches and for edge-triggered flip-flops) than with transition-sensitive non-return-to-zero clock lines, it is necessary to generate return-to-zero signals, $w_i$, from the transition signals $c_i$ . Since all $c_i$ signals are mutually exclusive and occur in a strict sequence of increasing indices, we can generate $w_i$ as below

$$w_i = merge(c_i, c_{i+1})$$

The counter *Cr* is used to generate the signals to select the next register $R_i$ to output via the multiplexer *MUX*. As the FIFO buffer has an active output end, i.e. *req.out* and *data.out* are sent first and then acknowledged, the first Muller-C element is one-hot and *Cr* starts in a state where the first output has already been produced. Reading must not overtake writing and the matching outputs of *Cw* and *Cr* are synchronised using Muller-C elements so that the data word in $R_i$ is selected and output only when the *(i-1)*-th reverse acknowledge signal on *ack.out* is matched with the $i$-th request signal on *req.in*. (All arithmetic is modulo-$N$.)

Similarly to above, it is much easier to construct multiplexers with level-sensitive control lines. Level-sensitive signals, $r_j$ , can be derived from the transition signals $e_j$. Again all ej are mutually exclusive and occur in a strict sequence of increasing indices, so

$$r_j = merge(e_j, e_{j+1})$$

A reverse acknowledge signal ack.in is generated when the FIFO is empty and a new burst can be accommodated. This occurs when the last word of the previous burst has been read out and acknowledged on *ack.out*, which is indicated by a transition on output 1 of *Cr*.

It will now become clear why the request signal req.in is stored and multiplexed with the data to generate the output request req.out, rather than by merging the transition signals $e_j$ together. The latter would require an N-input merge element which would increase the latency to much more than that of a register and a multiplexer. An implementation of the N-place counters is shown in Figure 9 below.

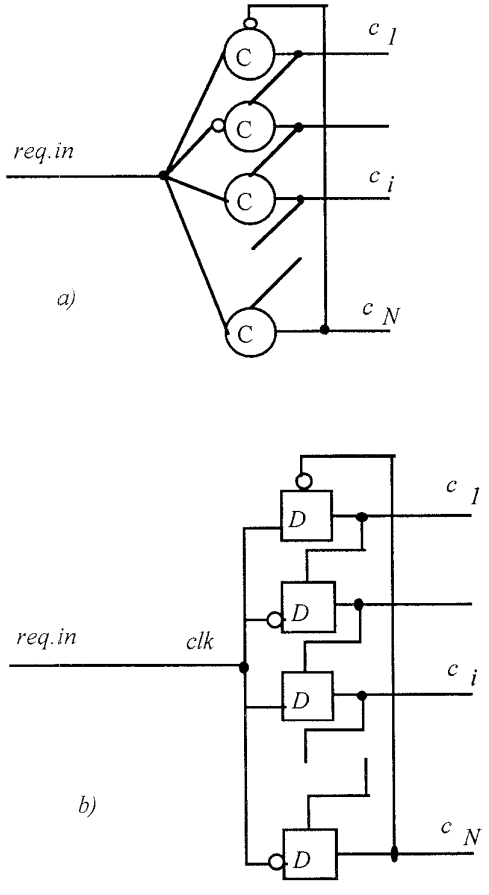



**Fig 9. *N*-place counters:**
**a) *N* odd;**
**b) *N* even.**



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

When $N$ is odd we need to use $N$ Muller-C elements and when it is even we need to use $N$ D-type latches. The D-Type latches must be carefully designed to prevent data racing through. An alternative design to the ring counter in Figure 9(b) which may also give superior performance is described in [11,12].

## 5 Interleaved Forward Transmission and Reverse Acknowledge Signals For Increased Throughput

There is a serious drawback to the burst transmission signalling scheme that we have considered so far. As every N-bit burst is acknowledged before the next one is sent and if the speeds/timings of the input and output ends of the FIFO are mismatched, then there may be considerable gaps in the transmission between successive bursts. A gap may be as long as the time to transmit a whole burst and this will reduce the link utilisation and, more importantly, the throughput. Even when the two ends of the FIFO are perfectly matched, consecutive bursts will always be separated by at least a gap proportional to the distance L, since the sender needs to wait for ack.in before sending the next burst.

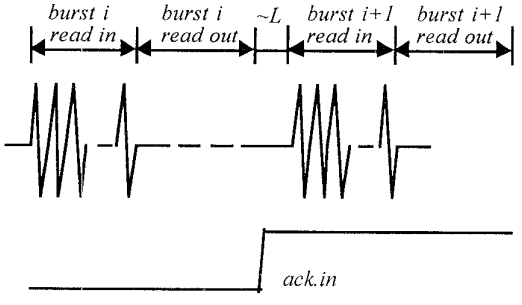



**Fig 10 Gaps in the transmission when waiting for the reverse acknowledge.**

The input and output ends of the FIFO can be matched better, and link throughput will be improved, if we allow for the interleaving of forward transmission of data and reverse acknowledge signals. This requires that the FIFO buffer be able to accommodate up to two bursts at any one time. The advantage comes when a new burst is sent to the FIFO while it is still outputting the previous one, Fig. 11.

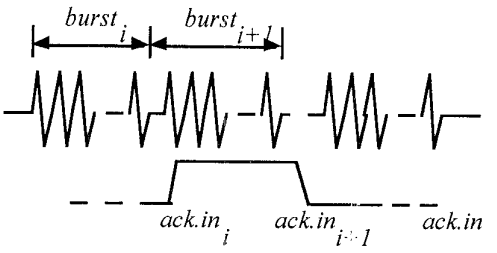



**Fig 11 Interleaved forward transmission/reverse acknowledges.**

The figure illustrates that bursts can now be much more closely packed together and that the link utilisation is much improved.

Extending the previous low-latency FIFO buffer design to allow for interleaved forward-transmission/reverse-acknowledge is quite straightforward. We need to stack up two parallel data paths, each of capacity N, and generate two acknowledge signals one after the first N data words have been output to the right and the other after the next N; the two acknowledge signals are then merged together. The modified control circuit is illustrated in Figure 12; the data-path is virtually unchanged but for the doubled capacity.

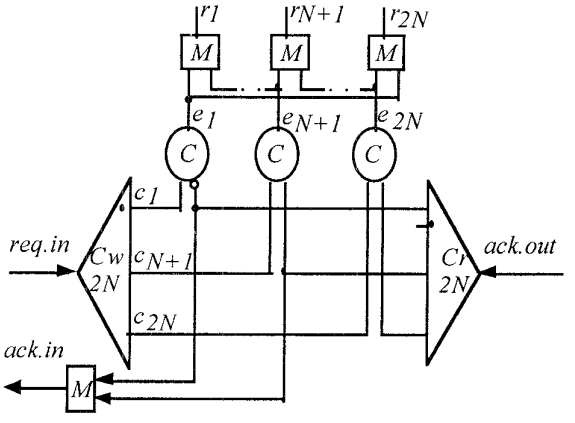



**Fig 12 Wagging-tail buffer of capacity *2N* - control circuit.**

From Figure 12 it is obvious that two consecutive bursts can be accommodated before any acknowledge signal ack.in is sent back. The first ack.in is generated after only N data words have been read out, i.e. one burst, the next one after the next N words are read out, and so on.

## 6 Performance Comparisons and Conclusions

In this paper we have considered the use of burst transmission of data between (packet switching) chips for increased throughput. Interleaved forward-



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

transmission/reverse-acknowledges have been shown to further improve the throughput. The major problem identified was the increase in latency or node-through delay incurred by the need to use FIFO buffers to accommodate one or more bursts. To reduce the latency a parallel FIFO buffer design has been proposed, which allows data words to be read out as soon as read in. This contrasts with pipeline FIFO designs, where each data word must propagate through all N stages of the pipeline before appearing at the output. The new design is expected to significantly reduce latency and detailed simulations have been carried out to evaluate the improvement and the costs, if any.

If we compare the pipeline FIFO buffer in Figure 4(a) with the parallel low-latency FIFO introduced in Section 4, it is apparent that they do not differ much in complexity. The low latency FIFO has a somewhat more complicated control circuit and uses an N-way multiplexer. However, in byte-wide transmission the bulk of the area is taken by the storage registers: the control circuit occupies only a small proportion of the total area. Table 1 below summarises the area and performance comparisons between the two designs. Each FIFO is of capacity 16 with a byte-wide data-path. Based on a 1 micron CMOS process, we used the Led layout editor and Lsim analog simulator tools of Mentor Graphics` GDT Designer.

| | *Area* | *Latency* | *Cycle time* |
|---|---|---|---|
| *pipeline FIFO* | *.92 x .79* | *14.00 nsec* | *.85 nsec* |
| *parallel FIFO* | *1.1 x .98* | *3.67 nsec* | *1.3 nsec* |
| *ratio paral/pipeline* | *1.53* | *0.26* | *1.55* |

**Table 1 Performance comparisons: parallel FIFO vs. pipeline FIFO.**

Most impressively, latency has been improved by a factor of four, at the expense of a much smaller increase in area. The increased cycle time is still much lower than the cycle time incurred by even the most advanced VLSI interchip signaling schemes.

The latency of the parallel FIFO changes very little when $N$ grows and the improvement over the pipeline FIFO becomes even more impressive:

| *No of stages* | *Improvement in latency* |
|---|---|
| *8* | *~2.2* |
| *16* | *~4.0* |
| *32* | *~7.0* |

A further important advantage of the parallel FIFO that we wish to point out is the reduction in power consumption. The power consumed by CMOS circuits is proportional to the circuit activity, i.e. the number of node signal-transitions, and the load on the nodes. IFor simplicitry assume first that the load of all nodes is the same. In a pipeline FIFO, where every word must propagate through all N stages of the pipeline, a burst of size N will cause power consumption proportional to N x N. The parallel design proposed here, where each data word must only propagate through one register and through one of the inputs of the multiplexer, will cause power consumption only proportional to N.

However, as mentioned before, CMOS power consumption is also proportional to the load of the nodes. One of the advantages of the micropipeline FIFO is that data nodes have low input capacitance. With the parallel FIFO inputs are forked to all storage cells and some of the nodes may have larger capacitances or alternatively an active fanout tree can be used. This will reduce the effect of node load on power consumption to a factor that is of the order of *(log N)*. Therefore, the overall power consumption of the parallel FIFO will compare to that of the micropipeline FIFO as

parallel FIFO ~ $N \log(N)$ pipeline FIFO ~ $N^2$

This improvement may be important in some cases.

## Acknowledgments

The authors would like to acknowledge the support of the European Comission under Basic Research Working Group 7225 ACiD-WG as part of the ESPRIT Third Framework, and of the UK SERC under grant GR/H77736.

## References

[1] JT Yantchev, CR Jesshope, "Adaptive, Low Latency, Deadlock-Free Packet Routing for Processor Networks," *IEE Proc. E,* May 1989.

[2] CR Jesshope, P Miller, JT Yantchev, "High Performance Communications in Processor Networks, " *Proc. 16-th Annual Int. Symp. on Comp. Arch, ISCA'16*, Israel, 1990.

[3] M Josephs, RH Mak, JT Udding, T Verhoeff, JT Yantchev, "High Level Design of an Asynchronous Packet Router Chip," In: J Staunstrup, R Sharp (eds) *Designing Correct Circuits. IFIP Transactions A-5*, North-Holland, 1992.

[4] JT Yantchev, I Nedelchev, "Implementation of Packet Switching Devices as Delay-Insensitive Circuits," *Proc. Research on Integrated Systems`93*, MIT Press, 1993.

[5] C. Seitz, WK Su, "A Family of Routing and



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.

communication Chips based on Mosaic," *Proc Research on Integrated Systems'93*, MIT Press, 1993.

[6] D May, *et al*, *Routers and Transputers*, IOS Press 1993.

[7] I Sutherland, "Micropipelines," *CACM* 32:6, (720-738), June 1989.

[8] C Mead, L Conway, *Introduction to VLSI Systems*, Reading, Mass.: Addison-Wesley, 1980, Chapter 7.

[9] K van Berkel, *Handshake Circuits: An asynchronous architecture for VLSI programming*, Cambridge International Series on Parallel Computation 5, Cambridge University Press, 1993.

[10] A Yakovlev, A Kolemans, L Lavagno, "High Level Modelling and Design of Asynchronous Interface Logic", *IEEE Design and Test of Computers*, Spring 1995.

[11] C. Farnsworth, "Low Powewr Implementations of an I2C I/O Expander," M.Sc Thesis, Dept of Computer Science, University of Manchester, UK.

[12] A Ashkinazy, D Edwards, C Farnsworth, G Gendel, S Sikand, "Tools for Validating Asynchronous Digital Circuits," *Proc ASYNC'94*, Salt Lake City, Utah, November, 1994.



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on August 24,2026 at 09:29:21 UTC from IEEE Xplore. Restrictions apply.